SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Code of Conduct:

I K. Lahari(192325118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Lahari

QUESTION 1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. **Identify appropriate HTML5 semantic elements missing in the structure.**
2. **Modify the structure to make it standards-compliant.**
3. **Interpret the HTTP request generated during form submission.**
4. **Analyze the HTTP response cycle from server to browser.**
5. **Suggest improvements for usability and accessibility.**

ANSWER :

1. Missing HTML5 semantic elements

Semantic elements describe the meaning of content, so that browsers, search engines and screen readers understand the page. The given snippet is only a bare form. The following are missing:

- **<!DOCTYPE html>, <html lang="en">, <head>, <meta charset>, <title>:** the document skeleton. Without a DOCTYPE the browser enters quirks mode, and without lang a screen reader may pronounce the text in the wrong language.
- **<header>:** holds the symposium name or logo and introductory heading.
- **<main>:** marks the primary content of the page (the registration form). Only one <main> is allowed per page and it lets screen-reader users jump straight to the content.

- **<section> (with a heading):** groups the registration part of the page as a thematic block.
- **<h1>/<h2> headings:** give the form a title and a clear document outline.
- **<fieldset> and <legend>:** group related form controls and describe the group.
- **<label>:** every input needs a programmatically linked label. This is the most important missing element.
- **<footer>:** holds contact details, copyright or a privacy-policy link.
- **<nav> (optional):** for navigation links such as Home, Schedule and Contact.

Other problems in the snippet: the <button> has no type, and the inputs have no id, required or autocomplete attributes.

2. Standards-compliant (corrected) structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>National Level Technical Symposium</h1>
  </header>

  <main>
    <section aria-labelledby="reg-title">
      <h2 id="reg-title">Student Registration</h2>

      <form action="/register" method="POST">
        <fieldset>
          <legend>Participant details</legend>

          <p>
            <label for="name">Full name</label>
            <input type="text" id="name" name="name"
                   autocomplete="name" required>
```

```
          </p>

          <p>
            <label for="email">Email address</label>
            <input type="email" id="email" name="email"
                   autocomplete="email" required>
          </p>

          <button type="submit">Register</button>
        </fieldset>
      </form>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 College Symposium Committee</p>
  </footer>
</body>
</html>
```

What was corrected: a valid document skeleton was added; semantic elements (header, main, section, footer) were added; each input now has a matching label for and id; required and autocomplete were added; and the button now has type="submit" and a meaningful text.

3. Interpreting the HTTP request during form submission

When the student clicks **Register**, the browser collects each control that has a name attribute, encodes the values and sends them to the URL in action, using the HTTP method in method. Here the method is **POST**, the target is **/register**, and the default encoding is application/x-www-form-urlencoded. If the student enters "Asha Kumar" and "asha@example.com", the request looks like this:

```
POST /register HTTP/1.1
Host: www.college.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 42
Origin: https://www.college.edu
Cookie: sessionId=abc123
User-Agent: Mozilla/5.0 ...

name=Asha+Kumar&email=asha%40example.com
```

- **Request line:** POST /register HTTP/1.1 gives the method, the resource path and the protocol version.
- **Headers:** Host identifies the server; Content-Type tells the server how to read the body; Content-Length is the size of the body in bytes; Cookie carries the session.
- **Blank line:** separates headers from the body.
- **Body:** name=Asha+Kumar&email=asha%40example.com is a list of name=value pairs joined by &. A space becomes + and @ becomes %40 (URL encoding).

Why POST? POST sends data in the body, not in the URL. It is meant for actions that change server state, such as creating a registration record, so it is correct here. It is not idempotent: submitting twice may create two records, so the server should check for duplicates. A GET request would put the data in the URL (/register?name=...&email=...), where it would be stored in history and logs.

4. HTTP response cycle from server to browser

The complete cycle has the following steps:

- **Connection:** the browser resolves the domain through DNS, opens a TCP connection and (for HTTPS) completes a TLS handshake.
- **Request sent:** the browser sends the POST request shown above.
- **Server processing:** the web server passes the request to the application, which validates the name and email, checks for duplicates, saves the data in the database, and decides on a response.
- **Response sent:** the server returns a status line, headers and (optionally) a body.
- **Browser handling:** the browser reads the status code. If it is a redirect, it makes a new GET request to the address in the Location header. If it is 200, it renders the HTML body.

A typical successful response uses the **Post/Redirect/Get** pattern, which prevents duplicate registration if the student refreshes the page:

```
HTTP/1.1 303 See Other
Location: /thank-you
Set-Cookie: sessionId=abc123; Secure; HttpOnly; SameSite=Lax
```

```
(browser then sends)
GET /thank-you HTTP/1.1
Host: www.college.edu

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html> ... "Thank you, registration successful" ...
```

Possible responses of the server are summarised below:

Status	**Meaning**	**Browser behaviour**
200 OK	Request processed; page returned	Renders the HTML (for example a confirmation page).
303 See Other	Success; go to another URL with GET	Automatically requests the URL in Location.
400 / 422	Invalid or missing data	Shows the form again with error messages.
403 Forbidden	Request rejected (for example invalid CSRF token)	Shows an error page.
409 Conflict	Email already registered	Shows a "already registered" message.
500 Server Error	Fault inside the server	Shows a generic error; nothing should be exposed.

5. Improvements for usability and accessibility (and security)

Usability

- Use required, type="email" and autocomplete so that the browser validates and auto-fills the data.
- Add short hints such as "Use your college email", using aria-describedby to connect the hint to its input.
- Use a clear button caption such as "Register for Symposium", not just "Submit".

- Display clear error messages next to the field and keep the values the student already typed.
- Show a confirmation page or email after successful registration.
- Make the layout responsive with the viewport meta tag so that the form is usable on mobile phones.

Accessibility

- Always connect label for with the input id. This enlarges the click area and lets screen readers announce the field name.
- Do not use the placeholder as a replacement for a label, because it disappears while typing and has low contrast.
- Use fieldset and legend for grouped controls, and a logical heading order (h1 then h2).
- Make sure the form can be used with the keyboard only (Tab, Enter) and has a visible focus outline.
- Keep colour contrast at least 4.5:1 and do not use colour alone to show errors.
- Announce validation errors with role="alert" or aria-live="polite".

Security

- Serve the page and the POST request only over **HTTPS**.
- Add a hidden **CSRF token**, and validate and sanitise all input again on the server (client-side checks can be bypassed).
- Use prepared statements to stop SQL injection and escape output to stop XSS.
- Add rate limiting or CAPTCHA to prevent bots from flooding the form, and mention a privacy policy because personal data is collected.

QUESTION 2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

Questions:

1. **Identify improper nesting and missing closing tags.**
2. **Analyze how different browsers may interpret this structure.**
3. **Identify missing semantic elements (e.g., header, section).**
4. **Provide a corrected W3C-compliant HTML structure.**
5. **Suggest techniques to ensure cross-browser compatibility.**

ANSWER 2:

1. Improper nesting and missing closing tags

- **<h1> is never closed.** The text "Welcome" has no </h1>.
- ** is never closed.** The paragraph "About us" has no . (HTML allows the browser to guess the end of a , but good practice and clean validation require an explicit end tag.)
- **Improper nesting:** because </h1> is missing, the starts while <h1> is still open. A paragraph is a block element and cannot be placed inside a heading, which accepts only inline (phrasing) content. The correct nesting is containing <h1> and as siblings.
- **Missing <!DOCTYPE html>** at the top of the page.

- **Missing lang attribute** on <html>.
- **Missing <meta charset="UTF-8">** and the viewport meta tag in <head>.

Wrongly nested version (what the parser sees):

```
<div>
  <h1>Welcome
    <p>About us      <-- p is inside h1 (invalid)
  </h1>
</div>
```

2. How different browsers may interpret this structure

HTML is forgiving. Browsers do not stop at errors; they use error-recovery rules to guess what the author meant. If the guesses differ, the layout differs.

- **Quirks mode:** with no DOCTYPE, browsers switch to quirks mode, which imitates very old browser behaviour. In this mode the box model, table font sizes, line heights and some margins are calculated differently in different browsers. This is the biggest cause of inconsistent layout in this snippet.
- **Auto-closing of tags:** modern browsers (Chrome, Edge, Firefox, Safari) follow the HTML5 parsing algorithm. They place the <p> inside the unclosed <h1> and then close both when they meet . As a result the text "About us" inherits the large, bold heading style, and it is also treated as part of the heading by screen readers and search engines.
- **Older or non-compliant browsers and tools** (for example old Internet Explorer versions or some email and embedded viewers) may close the <h1> before <p>, or ignore the paragraph margins, giving a different DOM tree and different spacing.
- **Different default styles:** each browser has its own default margins and font sizes for h1, p and body. Without a CSS reset, the same HTML looks slightly different in every browser.
- **Character encoding:** without <meta charset>, the browser guesses the encoding, and special characters may appear wrong.
- **Mobile scaling:** without the viewport meta tag, mobile browsers show the page zoomed out.

In short, valid markup gives every browser the same DOM tree, so it produces the same layout; invalid markup depends on each browser's own recovery rules.

3. Missing semantic elements

- **<header>:** for the site heading ("Welcome") and logo.
- **<nav>:** for the main menu (Home, About, Contact).
- **<main>:** to wrap the unique page content.
- **<section>:** for the "About us" content, with its own heading (<h2>).
- **<article>:** if there are independent items such as news or blog posts.
- **<footer>:** for copyright and contact details.
- A generic carries no meaning. It should be used only when no semantic element fits, for example as a styling wrapper.

4. Corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
    <nav aria-label="Main">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/about">About</a></li>
        <li><a href="/contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section aria-labelledby="about-title">
      <h2 id="about-title">About Us</h2>
```

```
      <p>About us</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Company Name. All rights reserved.</p>
  </footer>
</body>
</html>
```

Points to note: every opened tag is closed in the correct (last-opened, first-closed) order; the h1 appears once; headings follow a logical order (h1 then h2); and the document can be checked with the W3C Markup Validator (validator.w3.org) with no errors.

5. Techniques to ensure cross-browser compatibility

- **Write valid HTML:** always use <!DOCTYPE html>, close all elements, nest them correctly and validate the page with the W3C validator.
- **Use a CSS reset or normaliser** (for example normalize.css) so that default margins, fonts and form styles are the same everywhere. Also set box-sizing: border-box; globally.
- **Use modern layout** with Flexbox and CSS Grid instead of old float and table hacks, and use relative units (rem, %, vw) for flexible layouts.
- **Responsive design:** use the viewport meta tag and CSS media queries.
- **Progressive enhancement and feature detection:** build a working basic page first, then add advanced features. Use @supports in CSS (or feature checks in JavaScript) instead of checking the browser name.
- **Vendor prefixes:** use a tool such as Autoprefixer, driven by a Browserslist configuration, to add them automatically.
- **Check support** of features on caniuse.com or MDN before using them; provide fallbacks or polyfills where needed.
- **Fonts:** set a fallback font stack, for example font-family: "Segoe UI", Arial, sans-serif;.
- **Test on real browsers and devices** (Chrome, Firefox, Safari, Edge; Android and iOS) using developer tools or services such as BrowserStack, and run automated tests.

- **Avoid deprecated and non-standard tags and attributes** (<font>, , inline styling attributes).
- **Keep accessibility in mind:** semantic HTML works consistently with different browsers and assistive technology.

Small CSS example:

```
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: "Segoe UI", Arial, sans-serif; line-height: 1.5; }
main { display: flex; gap: 1rem; }
@supports not (display: flex) {
  main section { float: left; width: 100%; }   /* simple fallback */
}
```

QUESTION 3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. **Identify issues in the form configuration.**
2. **Analyze why the HTTP request is not triggered.**
3. **Interpret the role of GET method in this scenario.**
4. **Propose a corrected implementation.**
5. **Suggest improvements for secure data transmission.**

ANSWER 3:

1. Issues in the form configuration

- **type="button" on the Login button (main fault):** this is an ordinary button with no default behaviour. It does not submit the form, so no request is made.
- **method="GET" for a login form:** GET places the username and password in the URL, which is insecure.
- **No labels:** the inputs have no <label>, so screen-reader users do not know what to type, and the click area is small.
- **No id, required, or autocomplete attributes:** blank submissions are possible, and password managers cannot work properly.
- **No CSRF protection:** there is no anti-forgery token.
- **No HTTPS is enforced:** the action /submit is relative; it is safe only if the page itself is loaded over HTTPS.
- **No document structure:** the form is not placed within semantic elements and has no page title or lang attribute.

- **No JavaScript handler:** nothing in the snippet (such as an onclick event) sends the data by other means, such as fetch.

2. Why the HTTP request is not triggered

A browser sends form data only when a **submit event** occurs. This event is fired by a <button type="submit">, an <input type="submit">, a <button> without a type (default is submit), or by pressing Enter inside a text field when the form has a submit button. The sequence in this case is:

- The user types the username and password and clicks **Login**.
- The button has type="button", so the browser takes no action. It only generates a click event. Because no JavaScript is listening for that click, nothing happens.
- No submit event is fired, so the browser does not collect the data, does not build a URL or body, and does not open a connection to /submit.
- As a result, nothing appears in the Network tab, and the server never receives anything. The failure is on the client side, not the server side.

(There is one more possible cause: if the form had only a text input and a type="button" button, pressing Enter would also not submit it, since there is no submit button.)

3. Role of the GET method in this scenario

GET is designed to **retrieve** data. It is safe and idempotent: repeating it should not change anything on the server. The form data is added to the URL as a query string. If the button had been of type submit, the browser would have sent:

```
GET /submit?username=rahul&password=MySecret123 HTTP/1.1
Host: www.example.com
```

This causes the following problems for a login form:

- **Credentials are visible** in the address bar to anyone looking at the screen.
- **They are stored** in browser history, bookmarks, server and proxy logs, and may be leaked to other sites in the Referer header.
- **GET requests can be cached** and re-sent easily; the URL can be shared accidentally.
- **Length is limited** (URLs are typically limited to a few thousand characters).

- **Wrong semantics:** logging in creates a session, which changes server state. That should be done with POST, not GET.

GET is appropriate for searching, filtering and navigation, for example /search?q=html, where the data is not secret and the URL can be bookmarked. For login, **POST** should be used, which sends the data in the request body.

4. Corrected implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Login</title>
</head>
<body>
  <main>
    <h1>Login</h1>

    <form action="/submit" method="POST">
      <input type="hidden" name="csrf_token"
value="{{server-generated-token}}">

      <p>
        <label for="username">Username</label>
        <input type="text" id="username" name="username"
               autocomplete="username" required>
      </p>

      <p>
        <label for="password">Password</label>
        <input type="password" id="password" name="password"
               autocomplete="current-password" required>
      </p>

      <button type="submit">Login</button>
    </form>
  </main>
</body>
</html>
```

Changes made: the button is now type="submit"; the method is **POST**; labels and ids were added; required stops empty submissions; autocomplete values help password managers; a CSRF token was added; and a valid page skeleton was provided.

The browser will now send:

```
POST /submit HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 68

csrf_token=9f8a...&username=rahul&password=MySecret123
```

A correct server answer on success is a redirect with a secure session cookie:

```
HTTP/1.1 303 See Other
Location: /dashboard
Set-Cookie: sessionId=xyz789; Secure; HttpOnly; SameSite=Strict; Path=/
```

On failure, the server returns **401 Unauthorized** (or shows the form again with a general error message) as the response.

Alternative using JavaScript (if the button must stay type="button"): the handler has to send the data explicitly, but a normal submit button is simpler and works without JavaScript, so it is preferred.

```
<script>
  document.getElementById("loginBtn").addEventListener("click", async () =>
{
    const form = document.getElementById("loginForm");
    if (!form.reportValidity()) return;
    const response = await fetch("/submit", {
      method: "POST",
      body: new FormData(form),
      credentials: "same-origin"
    });
    if (response.ok) window.location.href = "/dashboard";
  });
</script>
```

5. Improvements for secure data transmission

- **Use HTTPS everywhere** (TLS) so that data is encrypted in transit. Redirect HTTP to HTTPS and enable **HSTS** (Strict-Transport-Security header).
- **Use POST, never GET,** for credentials, so that they stay out of URLs, history and logs.
- **CSRF protection:** use a per-session token and SameSite cookies.
- **Secure cookies:** set Secure, HttpOnly and SameSite flags on the session cookie, and regenerate the session ID after login.
- **Store passwords hashed on the server** with a slow, salted algorithm such as bcrypt, scrypt or Argon2, and never write passwords to logs.
- **Validate and sanitise on the server;** use prepared statements (against SQL injection) and output encoding (against XSS).
- **Limit login attempts** (rate limiting, temporary lock-out, CAPTCHA) to resist brute-force attacks.
- **Use generic error messages** such as "Invalid username or password" so that attackers cannot discover which usernames exist.
- **Prevent caching of sensitive pages:** send Cache-Control: no-store.
- **Add a Content-Security-Policy header** and keep scripts from untrusted sources out of the page.
- **Offer multi-factor authentication** and allow password managers (do not block paste in the password field).
- **Accessibility and usability:** keep labels, visible focus and clear error messages, so that all users can log in correctly.

Example of useful response headers on the server:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'
Cache-Control: no-store
X-Content-Type-Options: nosniff
```